

Functional Programming Lab 01: Simple Modulo Arithmetic

V. Markos
Mediterranean College

Fall, 2024 – 2025, Week 10

Abstract

This is a mini lab series quite relevant to concepts you will be assessed on for this semester on the Functional Programming module. We will explore expression parsing and evaluation in the context of a commutative ring, $\mathbb{Z}_n$, (a maths thing we will explain below), which is expected to help you make some connections with this semester's project.

1 Background: Modulo Arithmetic

Consider a positive integer, $n = 6$, let's say. Every number modulo 6 can be equivalent to any of the following (and just those):

$$0, 1, 2, 3, 4, 5.$$

Now, we define the following set:

$$\mathbb{Z}_6 = \{0, 1, 2, 3, 4, 5\},$$

and, on that set define addition as in Table 1:

+	0	1	2	3	4	5
0	0	1	2	3	4	5
1	1	2	3	4	5	0
2	2	3	4	5	0	1
3	3	4	5	0	1	2
4	4	5	0	1	2	3
5	5	0	1	2	3	4

Table 1: Addition modulo 6.

Essentially, what we have done above is to define addition between the elements of $\mathbb{Z}_6$ as addition modulo 6. So, more formally, $x + y$ in the context of

$\mathbb{Z}_6$ is defined as follows:

$$x + y \mod 6 = (x + y)\%6,$$

where we use the $\mod 6$ part to denote that any operation should be understood in $\mathbb{Z}_6$.

In a similar fashion, we can also define multiplication, as shown in Table 2.

$\cdot$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	1	2	3	4	5
2	0	2	4	0	2	4
3	0	3	0	3	0	3
4	0	4	2	0	4	2
5	0	5	4	3	2	1

Table 2: Multiplication modulo 6.

Again, this can be defined more succinctly as follows:

$$x \cdot y \mod 6 = (x \cdot y)\%6.$$

So, for instance:

- $7 + 15 \mod 6$ is $22 \mod 6$ which is $4 \mod 6$.
- $8 \cdot 3 + 4 \mod 12$ is $28 \mod 6$ which is, again, $4 \mod 6$.

For convenience we typically write just one $\mod$ at the end of each line (to avoid writing this all the time).

As you might have already observed, 6 is no more special than any non-trivial positive integer, $n > 1$. So, all the above can be generalised for any value of $n > 1$, as follows:

$$x + y \mod n = (x + y)\%n, \quad \text{and} \quad x \cdot y \mod n = (x \cdot y)\%n.$$

Also, as we often do with multiplication, we might omit $\cdot$ whenever this does not cause any confusion.

Now, we can prove (it's quite trivial in most cases) that the set $\mathbb{Z}_n = \{0, 1, \dots, n-1\}$ equipped with $+$ and $\cdot$ as its two operations is indeed a commutative ring with a unit, i.e., something that looks much like the set of real numbers equipped with its two operations as we know it¹. We shall denote this ring by $\mathbb{Z}_n$.

¹What we actually lack is a multiplicative inverse, since in $\mathbb{Z}_6$, for example, there exist non-zero elements, $x, y \in \mathbb{Z}_6$ such that $xy = 0$, e.g., $x = 2$ and $y = 3$. But you should not worry about that.

Alert 1: The Modulo Operator

Do not confuse the `mod` at the end of any lines of expressions / operations as in the above with the modulo operator, i.e., `%`. We will keep using `%` to denote the operation of extracting the modulus (remainder) of a number divided by another, while we shall use `mod` just as a reminder that `+` and `*` do not have their usual meaning, but their modular meaning!

2 Parsing $\mathbb{Z}_n$ Expressions

Our main task for this lab is to build a properly working expression parser and evaluator for valid expressions in the language of $\mathbb{Z}_n$. So, for instance, our parser should be capable of handling expressions as the ones below:

- $5 + 8 \bmod 7$.
- $4 * 8 + 4 \bmod 13$.
- $4 + 5 * (3 - 8) * (4 + 6 * 7) \bmod 21$.
- $(5 - 3 * (4 + 7 * (8 - 2))) + (5 - 6) \bmod 125$.
- $((((6))) + 4 \bmod 27$.

But, do not get terrified from the complexity of the above! We will start simple and see how we can abstract out any redundant complexity. So, here comes your first exercise for this lab.

2.1 Crafting Adequate Haskell Data Types

We have been talking quite a while about parsing a valid $\mathbb{Z}_n$ expression string, but, parser to *what*? Eventually, we should end up with a single parsing function as follows:

```
1 parse :: String -> ParseTree
```

So, as you might well suspect on your own, we have to somehow define this `ParseTree` data type in order to match our needs. To see how such a data type should look like, consider the following simple expression (not valid in $\mathbb{Z}_n$ but simple enough for our purpose):

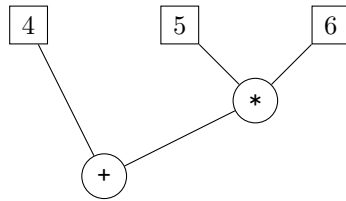
$$4+5*6 \tag{1}$$

As you already know, multiplication is of higher priority than addition, so this is something that should be reflected into our tree structure. But, how?

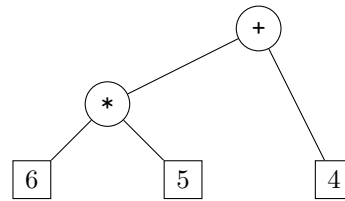
If you think of the above process step-by-step, what we would typically do is roughly as follows:

- We first spot the multiplication sign, $*$.
- We then execute this multiplication, i.e., we compute $5 \cdot 6$.
- Then we use this result to compute the sum we have to compute, i.e., $4 + 30$.

So, if you think of it as a pipeline of processes, this could be as shown in Figure 1a.



(a) How $4+5*6$ can be parsed.



(b) How $4+5*6$ can be parsed, just upside-down.

Figure 1: Parsing an expression as a tree.

Essentially, each expression, no matter how complex it might be, can be represented as a parse tree, much like the ones shown in Figure 1. Typically, we represent trees in Computer Science by placing their root node on top, so you should expect to see parse trees sketched as the one in Figure 1b, even if reading top-to-bottom might make more sense, as demonstrated in Figure 1a. However, you can think of them in any way suits you best!

As for the structure of such trees, the root node is typically the “last” operation to be performed, while each node might either encode an operation or be a number (modulo n). So, here comes your first exercise:

Exercise 2.1. You have to define any Haskell data types than can help you construct an expression parse tree. Take a look at Figures 1 and sketch a few trees of your own. For this exercise **ignore parentheses**, assuming that all expressions contain just the two operations, $+$, and $*$, and non-negative integers. Also, assume that all provided expressions are syntactically correct (no need to handle exceptions) and feel free to use `%` instead of `mod` to make things simpler when it comes to parsing.

Some examples you might want to test your implementations against:

- `3+5+87 mod 32 == 31`
- `5*4*7 mod 129 == 11`
- `5+4*6 mod 8 == 5`
- `2*1+4+6*9 mod 12 == 0`

Hint: You might define two data types, one to capture operations, `Operation`, and one to construct the parse tree, `ParseTree`. You might also define just one data type, which corresponds to a tree with operations as internal nodes and numbers as leaves. You might go a bit forward, defining your own number type — feel free to experiment on that! Also, how are you going to handle the modulus (%) that it is guaranteed to exist in every expression?

2.2 Building A Parse Tree

Now that we have some appropriate data types, we can utilise them to create an internal representation of a valid $\mathbb{Z}_n$ expression (ignoring parentheses). To that end, we have to actually build a tree from scratch based on a provided expression string, so, as before, we have to define a function `parse` as follows:

```
1 parse :: String -> ParseTree
```

The first to actually build our tree is to cleanse our input string a little bit, so here comes your next exercise:

Exercise 2.2. Create a Haskell function `tokenise :: String -> [String]` that splits a valid $\mathbb{Z}_n$ expression string into a list of its building tokens. Some examples to test your solution against:

- `tokenise "5 + 3 % 8" == ["5", "+", "3", "%", "8"]`
- `tokenise "13+7%9" == ["13", "+", "7", "%", "9"]`
- `tokenise "5 + 7 * 3 % 2" == ["5", "+", "7", "*", "3", "%", "2"]`

Hint: Think of what type of characters might a valid expression string contain and where you would need to split it to extract its building tokens. Also, `takeWhile` and `dropWhile` can be your best friends!

Now that you have properly split your initially messy expression string into a list of tokens, things should be much easier (or not). What you have to do is to parse this list of tokens in such a way that respects operator precedence. In case you are wondering how the modulus plays along with the rest, in our scenario, you can just treat % as an operator of the lowest priority, i.e., this will always be the root node of our tree. So, operator precedence in our case looks like below:

$$* \gg + \gg \%$$

In case you want to, you can also treat numbers as zero-arity operators, in which case precedence is defined as follows:

$$\text{number} \gg * \gg + \gg \%$$

In any case, here comes your next exercise:

Exercise 2.3. Define a Haskell function `parseTokens :: [String] -> ParseTree` which takes a list of tokens as its input, as produced by `tokenise` above, and returns a valid parse tree. Remember, you need not worry about parenthesisation at this very moment, since we are still working with very simple expressions. Also, feel free to ignore efficiency up to some certain extent!

Having completed the above, it is trivial to define a parser:

Exercise 2.4. Define a parser function, `parse :: String -> ParseTree` which takes a syntactically valid $\mathbb{Z}_n$ expression and outputs the corresponding parsing tree.

3 Evaluating Maths Expressions

Having built a parse tree, the only step remaining to complete your work is to actually have your tree evaluated. To that end, revisit Figure 1 and think how you would transverse this tree in order to obtain and evaluate the original expression? Once you have done that, you are ready for our next and final exercise:

Exercise 3.1. Define an evaluation function: `eval :: ParseTree -> Int` which takes a valid parse tree as an argument and computes the corresponding value. Some examples you might want to test your implementations against:

- `3+5+87 mod 32 == 31`
- `5*4*7 mod 129 == 11`
- `5+4*6 mod 8 == 5`
- `2*1+4+6*9 mod 12 == 0`

Hint: Since a `ParseTree` is a recursive data type, one way to define `eval` would be to try to pattern-match all the constructors you have defined for your `ParseTree` data type. In most cases, you just have to map operator nodes to actual Haskell maths operators.